

Transformer Modelleri ile Araç Hasar Tespiti Ve Model Eğitim Performans Karşılaştırması

Esat Berat Uzunca
221307044
Kocaeli Üniversitesi
Kocaeli-İzmit
Uzuncaesat@gmail.com

İbrahim Buğra San
221307059
Kocaeli Üniversitesi
Kocaeli-İzmit
ibugrasan@gmail.com

Abstract— Bu çalışmada, hasarlı araç fotoğraflarından dört farklı hasar türünü (çizik, göçük, cam kırığı, pert) tespit edebilen bir sistem geliştirilmiştir. Çalışmanın ilk aşamasında, web tarayıcıları (web crawler) kullanılarak bir veri seti oluşturulmuş, ardından veri temizleme işlemleri gerçekleştirilmiştir. Daha sonra, veri artırma teknikleriyle veri seti zenginleştirilmiştir. BEiT, Swin, DeiT, VGG16 ve ConvNext gibi modern derin öğrenme modelleri kullanılarak eğitim süreçleri gerçekleştirilmiştir. Her bir modelin doğruluk, kayıp ve sınıflandırma başarımı analiz edilmiştir. Çalışmanın sonuçları, önerilen yöntemlerin hasar tespitinde yüksek başarı sağladığını ortaya koymuştur.

Anahtar Kelimeler:Hasar tespiti, derin öğrenme, veri artırma, sınıflandırma, araç hasarı

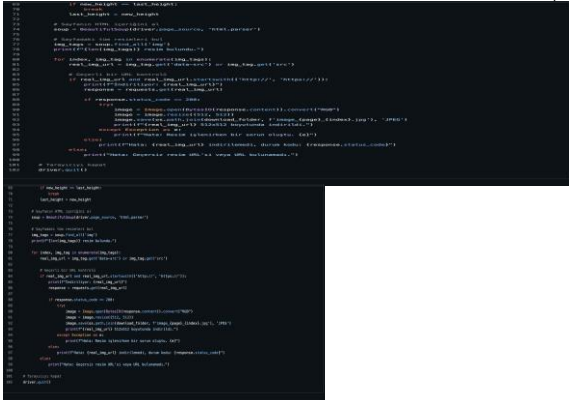
I. GİRİŞ

Araç hasar tespiti, sigorta, otomotiv ve ikinci el araç sektörlerinde önemli bir problemidir. Geleneksel yöntemler, insan gücüne dayalı olması nedeniyle zaman alıcı ve maliyetlidir. Bu çalışmanın amacı, otomatik araç hasar tespitine yönelik bir sistem geliştirmek ve modern derin öğrenme modelleri kullanarak doğruluğu yüksek bir sınıflandırıcı oluşturmaktır.

II. VERİ TOPLAMA VE TEMİZLEME

A. Veri Toplama

Çalışma kapsamında, hasarlı araç görüntülerini elde etmek için bir web crawler geliştirilmiştir. Farklı internet kaynaklarından toplamda 100.000'den fazla görüntü toplanmıştır. Görseller; çizik, göçük, cam kırığı ve pert olmak üzere dört sınıfa ayrılmıştır.



B. Maintaining the Integrity of the Specifications

Toplanan veriler, manuel süreçlerle temizlenmiştir. Anlamı olmayan veya sınıflandırma için uygun olmayan görseller elenmiştir. Sonuç olarak, 4.000 temizlenmiş ve etiketlenmiş görüntüden oluşan bir veri seti elde edilmiştir.

C. Veri Artırma

Veri setindeki çeşitliliği artırmak amacıyla Kontrast artırma, açı döndürme,, yansıtma, parlaklık değişimi teknikleri uygulanmıştır

III. KULLANILAN MODELLER

A. BEiT

BEiT, görsel dil modellemesi kavramını görüntü sınıflandırmaya taşıyan transformer tabanlı bir modeldir. Görüntüleri token'lara ayırarak metin tabanlı dil modellerine benzer şekilde işler. Maskelenmiş görüntü modelleme (Masked Image Modeling) yaklaşımı ile önceden eğitilmiş bu model, karmaşık görsel özellikleri anlamlandırmada oldukça başarılıdır. Bu çalışmada, BEiT modelinin güçlü temsil öğrenme kabiliyetleri kullanılmıştır.

B. SWİN

Swin Transformer, görsel veriler için özel olarak tasarlanmış bir hiyerarşik transformer mimarisidir. Shifted window (kaydırılmış pencere) dikkat mekanizması, hem yerel hem de küresel görsel ilişkileri öğrenme yeteneğini artırır. Swin Transformer, görüntülerin farklı çözünürlük seviyelerinde özellik haritaları oluşturabilmesi sayesinde çok ölçekli özellik öğreniminde başarılıdır. Bu nedenle, özellikle karmaşık hasar türlerinin ayırt edilmesinde etkili olmuştur.

C. DEiT

DeiT, veri etkinliği ile öne çıkan bir transformer modelidir. Görüntü sınıflandırma problemlerinde yüksek performans sunarken daha az veriyle etkili bir şekilde çalışabilmesi amacıyla geliştirilmiştir. Bu çalışmada, DeiT modeli hem veri etkinliği hem de hızlı eğitim süreci nedeniyle tercih edilmiştir.

D. VGG16

VGG16, derinliği artırılmış ve katmanların sabit filtre boyutlarına sahip olduğu bir CNN mimarisidir. Klasik bir model olarak, derin öğrenme alanında sağlam bir temel oluşturur. Hasar tespitine yönelik bu çalışmada, VGG16'nın görsel özellik çıkarımı konusundaki başarısı test edilmiştir. Basitliği ve tutarlılığı sayesinde, diğer modeller için karşılaştırma noktası olarak kullanılmıştır.

E. ConvNext

ConvNext, modern CNN mimarilerinin bir temsilcisi olarak geliştirilmiştir. Geleneksel CNN'lerin avantajlarını

korurken, transformer modellerinin getirdiği yeniliklerden de faydalanır. Özellikle, daha geniş kernel boyutları ve normalize edilmiş aktivasyon fonksiyonları kullanılarak performansı artırılmıştır. ConvNext, hem yüksek doğruluk oranları hem de verimli bir eğitim süreci sunmuştur.

IV. EĞİTİM SÜRECİ

Eğitim süreci, derin öğrenme modellerinin belirlenen hasar sınıflarını (Çizik, Göçük, Cam Kırığı, Pert) ayırt etme kabiliyetini optimize etmek amacıyla dikkatlice tasarlanmıştır. Bu süreç aşağıdaki adımları içermektedir:

A. Model Seçimi Ve Hazırlık

Hugging Face platformu üzerinden beş farklı model indirilmiş ve bu modeller eğitime sokulmuştur. Modelin son sınıflandırıcı katmanı (classifier), dört sınıfa uygun olacak şekilde yeniden düzenlenmiştir.

```
# Swin Tiny modelini yükleyelim
model_name = 'microsoft/swin-tiny-patch4-window7-224'
model = SwinForImageClassification.from_pretrained(model_name, num_labels=4, ignore_mismatched_sizes=True)
model.classifier = torch.nn.Linear(model.classifier.in_features, 4)
```

B. Veri İşleme

Görüntüler, torchvision.transforms modülü kullanılarak eğitim için ön işleme tabi tutulmuştur: Görüntü boyutları 224x224 olarak yeniden boyutlandırılmıştır. Normalize işlemi yapılmış, RGB kanallarına ait ortalama ve standart sapma değerleri kullanılarak normalizasyon sağlanmıştır.

```
# Veri yükleme kısmı (Google Drive üzerinde verinizi yükleyin)
drive.mount('/content/drive')

# Veriyi burada uygun şekilde yükleyin (örneğin, 'all_data' klasörü altında)
full_data = torchvision.datasets.ImageFolder('/content/drive/MyDrive/YazLabProjesi/DATABASE', transform=transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
]))
```

C. Veri Ayrımı

Eğitim ve test verileri, random_split fonksiyonu ile rastgele bir şekilde ayrılmıştır: %80'i eğitim için, %20'si test için kullanılmıştır. Veri yükleyiciler (DataLoader) kullanılarak veriler batch_size=16 olacak şekilde işlenmiştir.

```
# Eğitim ve test verisi ayırma işlemi (Veriyi %80 eğitim, %20 test olarak ayıralım)
train_size = int(0.8 * len(full_data))
test_size = len(full_data) - train_size
train_data, test_data = torch.utils.data.random_split(full_data, [train_size, test_size])

train_loader = torch.utils.data.DataLoader(train_data, batch_size=16, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=16, shuffle=False)
```

D. Eğitim Ayarları

Eğitim işlemleri için Adam optimizasyon algoritması ve CrossEntropyLoss kayıp fonksiyonu kullanılmıştır. Öğrenme oranı 1e-5 olarak belirlenmiştir. Eğitim süreci, GPU destekli bir ortamda (cuda) gerçekleştirilmiştir.

```
# 2. Eğitim Ayarları
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-5)
```

E. Performans Metrikleri

Doğruluk (Accuracy), Duyarlılık (Sensitivity), Özgüllük (Specificity), Kesinlik (Precision), F1 Skoru, ve **ROC AUC Skoru** gibi metrikler değerlendirilmiştir. Her epoch sonunda **Karmaşıklık Matrisi** (Confusion Matrix) ve **ROC Eğrileri** oluşturulmuştur.

```
# ROC eğrisini hesapla ve çiz
plot_roc_curve(all_labels_one_hot, all_outputs_softmax, lb.classes_, epoch)

# Accuracy, Recall, Precision, F1, AUC hesaplama
accuracy = accuracy_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds, average='weighted')
precision = precision_score(all_labels, all_preds, average='weighted')
f1 = f1_score(all_labels, all_preds, average='weighted')
auc_value = roc_auc_score(all_labels, all_outputs_softmax, multi_class='ovr', average='weighted')

accuracies.append(accuracy)
recalls.append(recall)
precisions.append(precision)
f1_scores.append(f1)
aucs.append(auc_value)
```

F. Eğitim Süreci

Her epoch için eğitim ve doğrulama aşamaları ayrı ayrı gerçekleştirilmiştir. Eğitim sırasında, modelin tahmin performansı sürekli izlenmiş ve kaydedilmiştir. Tüm sınıflar için tahmin edilen doğruluk ve olasılıklar torch.softmax yöntemiyle elde edilmiştir. Her epoch sonunda, modelin performansına dair detaylı raporlamalar yapılmış ve görselleştirilmiştir

```
def train_model(model, train_loader, test_loader, epochs=10):
    train_losses, test_losses = [], []
    accuracies, recalls, precisions, f1_scores, aucs = [], [], [], [], []
    confusion_matrices = []
    epoch_times, inference_times = [], []
    epoch_train_losses = [] # Train loss for each epoch
    epoch_test_losses = [] # Test loss for each epoch

    print("Model eğitime başlıyor...")

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        start_time = time.time()

        # Eğitim
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs).logits
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        epoch_time = time.time() - start_time
        epoch_times.append(epoch_time)

        epoch_train_losses.append(running_loss / len(train_loader))
```

G. Epoch Süreleri

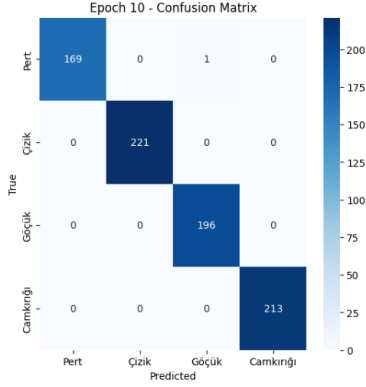
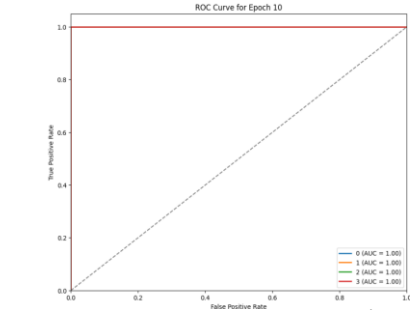
Her epoch başına eğitim süresi ve modelin doğrulama verisi üzerindeki çıkarım süresi detaylı olarak ölçülmüştür. Eğitim süreci boyunca toplamda 10 epoch tamamlanmış ve her bir epoch süresince modelin doğruluk ve kayıp metrikleri optimize edilmiştir.

```
epoch_time = time.time() - start_time
epoch_times.append(epoch_time)

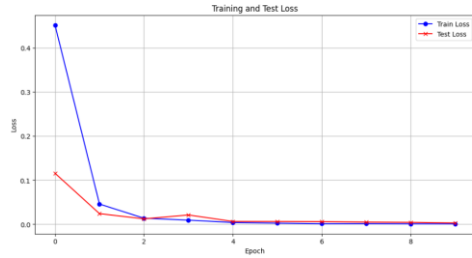
epoch_train_losses.append(running_loss / len(train_loader))
```

H. Sonuç

Eğitim sonunda, 5 modelin de genel doğruluk, kesinlik, duyarlılık, özgüllük, ve AUC skorları değerlendirilmiştir. Performans sonuçları ve metrikler, detaylı analiz için raporlanmıştır.



Epoch 10/10
Train Loss: 0.0008
Accuracy: 0.9988, Recall: 0.9988, Precision: 0.9988
Sensitivity: [1.0, 1.0, 0.9983443708609272, 1.0], Specificity: [0.9941176470588236, 1.0, 1.0, 1.0]
F1 Score: 0.9987, AUC: 1.0000
Epoch Time: 53.25s, Inference Time: 6.71s



V. DENEYSEL SONUÇLAR

A. Model Performansları

Her epoch başına eğitim süresi ve modelin doğrulama verisi üzerindeki çıkarıma

Beit: Accuracy: 0.9973, Recall: 0.9975, Precision: 0.9963, F1-Score: 0.9969, AUC: 0.9983 Training Loss: 0.0026, Validation Loss: 0.0072

ConvNext: Accuracy: 0.9938, Recall: 0.9938, Precision: 0.9938, F1-Score: 0.9937, AUC: 0.9958 Training Loss: 0.0005, Validation Loss: 0.0162

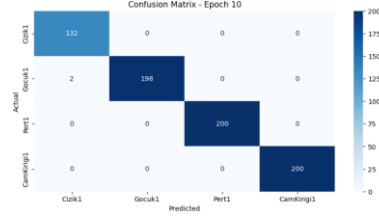
VGG16:Accuracy: 0.9487, Recall: 0.9487, Precision: 0.9525, F1-Score: 0.9490, AUC: 0.9658 Training Loss: 0.0206, Validation Loss: 0.1644

SwinTiny: Train Loss: 0.0008 Accuracy: 0.9988, Recall: 0.9988, Precision: 0.9988 Sensitivity: [1.0, 1.0, 0.9983443708609272, 1.0], Specificity: [0.9941176470588236, 1.0, 1.0, 1.0] F1 Score: 0.9987, AUC: 1.0000 Epoch Time: 53.25s, Inference Time: 6.71s

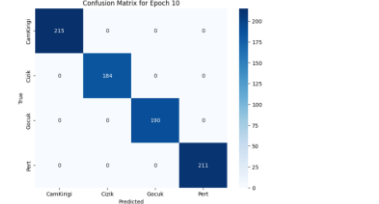
Deit: Accuracy: 0.9421, Recall: 0.9418, Precision: 0.9465, F1-Score: 0.9439, AUC: 0.9604, Training Loss: 0.0254, Validation Loss: 0.1587

B. Karmaşıklık Matrisleri

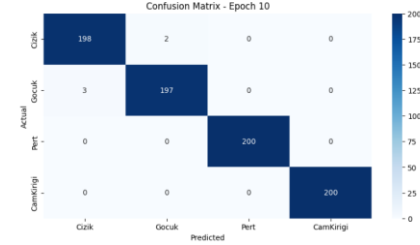
Beit:



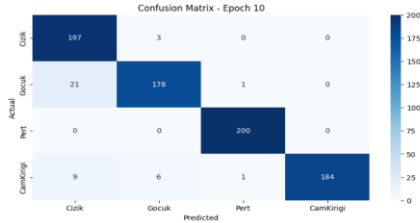
Deit:



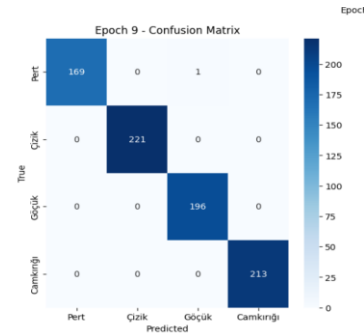
ConvNext:



VGG16:



SwinTiny:



VI. SONUÇ

Bu çalışma, hasarlı araç fotoğraflarından hasar tespiti yapmak için derin öğrenme modellerinin uygulanabilirliğini göstermiştir. BEiT ve Swin Transformer modelleri, yüksek doğruluk oranları ile öne çıkmıştır. Gelecekteki çalışmalar, gerçek zamanlı hasar tespiti ve daha geniş bir veri seti ile model performanslarının iyileştirilmesini hedeflemektedir.

REFERENCES

- [1] <https://huggingface.co/search/full-text?q=facebook%2Fdeit-base-distilled-patch16-224>
- [2] <https://huggingface.co/Felix92/doctr-dummy-torch-vgg16-bn-r>.
- [3] <https://huggingface.co/microsoft/beit-base-patch16-224>
- [4] <https://huggingface.co/microsoft/swin-tiny-patch4-window7224>
- [5] <https://huggingface.co/facebook/convnext-base-224>
- [6] <https://pytorch.org/docs/stable/index.html>
- [7] https://www.tensorflow.org/api_docs .